МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: «Расстояние Левенштейна»

Студентка гр. 4345 _____ Бодарева С.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы.

Изучить и реализовать алгоритм поиска расстояния Левенштейна. Провести исследование его работы на различных размерах входных данных.

Задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.
- Затем необходимо заменить два последних символа: stal -> stie.
- Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. $(S, 1 \leq |S| \leq 2550)$.

Вторая строка входных данных содержит строку из строчных латинских букв. $(T, 1 \leq |T| \leq 2550)$.

Параметры выходных данных:

Одно число L, равное расстоянию Левенштейна между строками S и T.

Sample Input:

pedestal

stien

Sample Output:

7

Вариант 1.

"Особо заменяемый символ и особо добавляемый символ": цена замены определённого символа отличается от обычной цены замены; цена

добавления другого (или того же) определённого символа отличается от обычной цены добавления. Особо заменяемый символ и цена его замены, особо добавляемый символ и цена его добавления — дополнительные входные данные.

Основные теоретические положения.

Расстояние Левенштейна – метрика, измеряющая по модулю разность между двумя последовательностями символов. Она определяется как минимальное количество односимвольных операций (вставка, удаление, замена), необходимых для превращения одной последовательности символов в другую.

Расстояние Левенштейна и его обобщения применяется:

- Для исправления ошибок в слове (в поисковых системах, базах данных, при вводе текста, при автоматическом распознавании отсканированного текста или речи).
- Для сравнения текстовых файлов утилитой diff и ей подобными. Здесь роль «символов» играют строки, а роль «строк» - файлы.
- В биоинформатике для сравнения генов, хромосом и белков.

Пусть S_1, S_2 – две строки (длиной M и N соответственно) над некоторым алфавитом, тогда редакционное расстояние $d(S_1, S_2)$ можно подсчитать по следующей рекуррентной формуле:

$d(S_1, S_2) = D(M, N)$, где

$$D(i,j) = \begin{cases} 0, & i = 0, j = 0 \\ i, & i > 0, j = 0 \\ j, & i = 0, j > 0 \\ \min \begin{matrix} \{D(i, j-1) + 1, \\ D(i-1, j) + 1, \\ D(i-1, j-1) + \mathrm{m}(S_1[i], S_2[j])\} \end{matrix} & i > 0, j > 0 \end{cases}$$

Где $\mathrm{m}(a, b)$ равна нулю, если $a = b$ и единице в противном случае.

Выполнение.

Реализация алгоритма вычисления расстояния Левенштейна с использованием таблицы размера $M \times N$ требует $O(MN)$ операций и столько

же памяти. Для длинных строк такие затраты памяти могут стать проблемой, поэтому для вычисления редакционного расстояния без поиска редакционного предписания вместо всей таблицы можно рассматривать только две строки, так как после вычисления любой строки предыдущая больше не нужна, затраты памяти станут $O(\min(N, M))$. На первом шаге предыдущая строка *previous* будет состоять из чисел от 0 до n, на каждом шаге первым числом в строке *current* будет i – индекс текущей буквы в строке 1. Текущее *current[j]* будет минимумом из следующих значений:

- *current[j-1]+insert_cost*;
- *previous[j]+delete_cost*;
- *previous[j-1]+replace_cost*;

После заполнения строки по заданному правилу, текущая строка становится предыдущей, а новая текущая заполняется заново по тому же принципу. Последнее число в последнее строке и будет редакционным расстоянием между заданными строками.

В соответствии с **вариантом 1** алгоритм был модифицирован. При вводе можно задать цену замены для определенного символа и цену и добавления другого (или того же) определенного символа. Для этого после ввода строк S и T нужно ввести в разных строках два значения: особый символ и его цену сначала для замены, потом для добавления, эти строки можно оставить пустыми, тогда значениями по умолчанию будут пустая строка и 1.

Исследуем временную сложность алгоритма.

Результаты исследования см. в Таблице 1.

Таблица 1. Время работы алгоритма

Длина строк S и T	Время, с
10	0.000064
100	0.003
500	0.113
1000	0.544
1500	1.389
2000	2.595
2500	3.822

График зависимости времени вычисления редакционного расстояния от длины строк см. на Рис. 1.

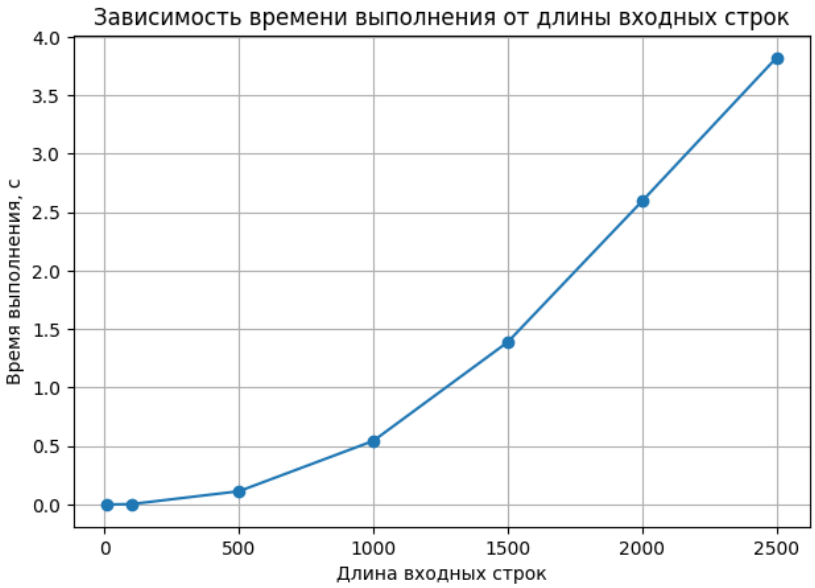

Рисунок 1. График времени вычисления редакционного расстояния от длины входных строк.

Временная сложность алгоритма $O(M \times N)$, оптимизация затрат памяти не оптимизирует затраты по времени выполнения. Кроме того, время выполнения зависит только от длины входных строк и не зависит от того, насколько различны поданные на вход строки.

Исходный код программы см. в Приложении А.

Результаты тестирования см. в таблице 2.

Таблица 2. Результаты тестирования

№	Входные данные	Выходные данные	Комментарии
1	pedestal stien	7	Пример из задания.
2	stroka stroka	0	Совпадающие строки
3	aaaa bbbb	4	Полностью различные строки
4	boba bomba m 4	2	Цена вставки символа m = 4 вместо 1, поэтому вместо одной вставки было произведено две операции: замена b на m и вставка b

5	present prevent s 10	2	Цена замены символа s = 10, поэтому вместо замены происходит 2 операции – удаление s и вставка v

Вывод.

Был изучен и реализован алгоритм вычисления расстояния Левенштейна. Были произведены исследования времени выполнения алгоритма для разных длин входных строк. Получившаяся временная сложность – $O(M \times N)$ совпадает с теоретической оценкой. Затраты памяти $O(\min\{M, N\})$ после оптимизации, где M и N – длины входных строк. Была проведена модификация алгоритма – добавлена возможность задавать особые цены вставки и замены для выбранного символа.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: main.py

```
import time
import random
import string



def replace_cost(ch1: str, ch2: str,  special_ch: str, special_cost: int) ->
int:
    if ch1 == ch2:
        return 0
    if ch1 == special_ch:
        return special_cost
    return 1

def insert_cost(cur_ch: str, special_ch: str, special_cost: int) -> int:
    if cur_ch == special_ch:
        return special_cost
    return 1

def LevenshteinDistance(str1: str, str2: str, ch_replace: str, replace: int,
ch_insert: str, insert: int) -> int:
    m = len(str1)
    n = len(str2)
    previous = [x for x in range(n+1)]
    current = [0] * (n + 1)
    for i in range(1, m+1):
        current[0] = i
        for j in range(1, n+1):
            current[j]  =  min(current[j-1]+insert_cost(str2[j-1],  ch_insert,
insert),
                              previous[j]+1,
                              previous[j-1] + replace_cost(str1[i-1], str2[j-1],
ch_replace, replace))
        previous, current = current, previous
    return previous[n]


S = [random.choice(string.ascii_letters) for _ in range(2500)]
T = [random.choice(string.ascii_letters) for _ in range(2500)]
#S = list(input())
#T = list(input())
ch_r, r_cost = '', 1
ch_i, i_cost = '', 1
replace_input = input()
if replace_input:
    ch_r, r_cost = replace_input.split()
    r_cost = int(r_cost)

insert_input = input()
if insert_input:
    ch_i, i_cost = insert_input.split()
    i_cost = int(i_cost)

start = time.time()
L = LevenshteinDistance(S, T, ch_r, r_cost, ch_i, i_cost)
finish = time.time()
print(L, finish-start)
```